Homework #3

CE 6320: Applied Data Structures and Algorithms (Fall 2026)

Department of Electrical and Computer Engineering
University of Texas at Dallas

Instructor: Prof. Mohamed Ibrahim
Due Date: October 20, 2026

Student Name: _____

Student ID: _____

Instructions:

- This homework has **5 problems**.
- You are required to **solve and submit all 5 problems**.
- Each chosen problem will be graded out of its assigned points; your total will be scaled to an overall maximum of **100 points**.
- Show all steps clearly. Partial credit will be awarded.

October 8, 2026

Problem 1: Merge K Sorted Lists with a Min-Heap [20 points]

You are given K sorted singly linked lists (ascending) and must merge them into a single sorted list using a **Min-Heap of list-node pointers**. The heap stores non-null node pointers, ordered by the node's key $p \to val$. The merge repeatedly pops the smallest node from the heap, appends it to the output list, and if that node has a successor, pushes the successor to the heap.

Illustrative example (trace table). Consider $K = 3$ with lists $[1, 4, 5]$, $[1, 3, 4]$, $[2, 6]$. At each step, the heap contains the current "frontier" nodes (the current heads of the remaining suffixes). On pop, the chosen node from heap is appended to the output list; if it has a next, that next is pushed to the heap.

Step	List heads (values or $\varnothing$)	Heap	Popped	Output list
Init	$(1, 1, 2)$	$\{1, 1, 2\}$	–	[ ]
1	$(4, 1, 2)$	$\{1, 2, 4\}$	1 (from list 1)	[1]
2	$(4, 3, 2)$	$\{2, 3, 4\}$	1 (from list 2)	$[1, 1]$
3	$(4, 3, 6)$	$\{3, 4, 6\}$	2 (from list 3)	$[1, 1, 2]$
4	$(4, 4, 6)$	$\{4, 4, 6\}$	3 (from list 2)	$[1, 1, 2, 3]$
5	$(4, 4, 6)$	$\{4, 4, 6\}$	4 (from list 1 or 2; ties allowed)	$[1, 1, 2, 3, 4]$
$\vdots$	Continue until all lists are exhausted			

1. Diagram (5 pts). Use $K = 4$ with lists $[0, 7]$, $[1, 4, 5]$, $[1, 3, 4]$, $[2, 6]$.

1. Draw the Min-Heap *as an array and as a conceptual tree* immediately after pushing the four initial heads.
2. Provide a *step table* (same format as above) for the first **5** pop/append operations, showing the heap content and the output prefix at each step.
3. Redraw the heap array after the 3rd and after the 5th operation.

---

2. C++ Program (9 pts). Complete the bodies of the following items. Do not add extra global state beyond what is declared. Functionality must match the algorithm described above.

```
struct Node {
    int   val;
    Node* next;
};

/* ---------- Min-Heap of Node* keyed by pointer->val ---------- */
extern Node* heap[];   // heap[0..heapSize-1]
extern int   heapSize; // current number of elements

/* Swap two entries at indices i and j in heap[] */
void hSwap(int i, int j);

/* Return parent/children indices for a given i */
int hParent(int i);  // (i - 1) / 2
int hLeft  (int i);  // 2*i + 1
int hRight (int i);  // 2*i + 2

/* Push non-null pointer p into the min-heap */
void hPush(Node* p);
```

```
/* Pop from heap and return the smallest pointer; return nullptr if empty */
Node* hPop();

/* Heapify-down from index i within current heapSize */
void hDown(int i);

/* Heapify-up from index i within current heapSize */
void hUp(int i);

/* ---------- Merge driver ---------- */
/* Given heads[0..K-1], return head of merged list */
Node* mergeK(Node* heads[], int K);

/* ---------- Minimal driver template (for understanding call flow) ----------
   This template is NOT a solution and may not compile as-is.
   It only indicates how the above functions are expected to be used.
   You do not need to fill out this main function.

int main() {
    // 1) Build some lists (e.g., from static arrays) -> Node* L0, L1, ...
    // 2) Place their heads into an array: Node* heads[K] = {L0, L1, ...};
    // 3) Initialize heapSize = 0; for each heads[i] != nullptr, call hPush(heads[i]);
    // 4) Call: Node* merged = mergeK(heads, K);
    // 5) Optionally print merged list and free all nodes.
    // return 0;
}
---------------------------------------------------------------------- */
```

---

3. Complexity (3 pts).

1. Give the asymptotic time to merge N total nodes from K lists using this heap approach.
2. State the extra space used by the heap in terms of K.

---

4. Discussions (3 pts).

1. **Exact operation counts.** Let N be the total number of nodes and let L_0 be the number of non-empty lists at initialization. Assuming you push exactly the initial non-null heads and thereafter push the successor of each popped node when it exists, give exact closed-form expressions (in N and L_0) for the total number of calls to hPush and to hPop over the entire merge.
2. **Deterministic stability.** With comparison key p $\rightarrow$ val only, equal values from different lists need not preserve cross-list order. Specify a deterministic, implementable tie-breaking rule (expressible via information already available to the algorithm) that guarantees *stable* cross-list ordering, and justify in 2–3 precise sentences why it enforces stability.

Problem 2: Heapify a Subtree in Max-Heap **[20 points]**

You are given a **Max-Heap** $H = [90, 70, 85, 40, 60, 30, 80]$ (array index starting at 0). Suppose the value at index 2 is accidentally reduced to 10, so the array becomes $[90, 70, 10, 40, 60, 30, 80]$. Your goal is to restore the heap by **heapifying down from index 2** only, i.e., repeatedly compare the node at the current index with its two children (if present), swap it with the larger child if needed, and continue at that child's index until the Max-Heap property holds.

Illustrative trace table for this instance. The table shows the state at each step of `maxHeapifyFrom(2)`. Children of i are $2i + 1$ (left) and $2i + 2$ (right).

Step	Array (snapshot)	Index i	Children (values)	Action	Result
Init	[90, 70, 10, 40, 60, 30, 80]	2	left=30, right=80	swap ...	[90,70,80,40,60,30,10]
1	[90, 70, 80, 40, 60, 30, 10]	6	no children (leaf)	stop	[90,70,80,40,60,30,10]

This restores the Max-Heap property in the sub-tree rooted at index 2 and therefore in the whole heap.

---

1. Diagram (5 pts). Use $H = [95, 70, 85, 40, 60, 30, 80, 20]$. First, set the value at index 1 to 10, then perform `maxHeapifyFrom(1)`.

1. Draw the heap (array and tree) immediately before calling `maxHeapifyFrom(1)`.
2. Provide a *step table* (same format as above) for each swap/advance of the index during `maxHeapifyFrom(1)`.
3. Draw the final heap (array and tree) after `maxHeapifyFrom(1)` completes.

---

2. C++ Program (9 pts). Complete the bodies of the following functions. Do not add extra global state beyond what is declared by you in this file. Behavior must match the algorithm described above.

```
/* Restore Max-Heap property by heapifying down from index i only.
   Arguments: array C[0..cN-1], heap size cN, start index i. */
void maxHeapifyFrom(int C[], int cN, int i);

/* Return 1 if Max-Heap property holds at index i (i.e., children <= C[i]),
   otherwise return 0. Only checks local condition at i. */
int holdsAt(int C[], int cN, int i);

/* Return 1 if the entire array C[0..cN-1] satisfies the Max-Heap property,
   otherwise return 0. */
int isMaxHeap(int C[], int cN);

/* (Optional helpers you may define and use)
   int leftChild(int i);
   int rightChild(int i);
   int largerChildIndex(int C[], int cN, int i); // -1 if no children
   void swapInt(int* a, int* b);
*/

int main() {
```

```
    int C[] = {90, 70, 10, 40, 60, 30, 80}; // example from statement
    int n = 7;
    // call: maxHeapifyFrom(C, n, 2);
    // verify: isMaxHeap(C, n);
    // print the final array C
    return 0;
}
```

---

3. Complexity (3 pts).

1. Give the worst-case time of `maxHeapifyFrom` on a heap of size n (asymptotic in n).
2. Let the heap be complete with height h (root at height 0). Starting from index 2 in the original instance, what is the maximum number of swaps that can occur as a function of the number L of levels descended (edges traversed) during heapify-down? Give a tight bound in terms of L.

---

4. Discussions (3 pts).

1. **Exact index path.** For the instance in the statement ($[90, 70, 10, 40, 60, 30, 80]$ and start at $i = 2$), list the exact sequence of indices visited (including the start and the final resting index). Your answer must be a sequence of integers (unique).
2. **Key-comparison count.** Consider an implementation of `maxHeapifyFrom` that at each step compares the two children once to pick the larger child (if both exist), then compares the parent once to that larger child, and stops as soon as the parent is $\geq$ the larger child or a leaf is reached. If the procedure descends exactly L levels, give a tight upper bound on the *number of key comparisons* performed as a function of L. (Your bound should be a closed-form expression.)

Problem 3: Hybrid Hashing **[20 points]**

You will design and analyze a hybrid hashing scheme. There are B buckets; each bucket stores up to $c = 2$ keys *inline* (a small fixed array). If a bucket overflows, the extra keys go into a single **global linear-probing overflow area (OFA)** whose size is $\Theta(B)$. We use $h(x) = x \bmod B$ to choose a home bucket.

Placement rules. When inserting a key x:

1. Compute $b = h(x)$. If the bucket b has a free inline slot, place x there.
2. Otherwise, place x into the OFA via *linear probing* on an OFA array of length OF: start probing at index $\text{ofStart}(x) = h(x) \bmod OF$, advance by $+1$ modulo OF until an empty slot is found. (Use a tombstone for deletes.)

Lookups first scan the two inline slots of the home bucket, then (if not found) probe in the OFA starting at $\text{ofStart}(x)$ until either the key is found or an empty slot is encountered.

Example. Let's demonstrate the procedure using keys $[23, 43, 13, 27, 5, 25, 15, 35]$ with $B = 7$, $c = 2$, $OF = 11$. The table below tracks each insertion, the bucket index, whether it goes inline or to OFA (with the first OFA index used), and a compact view of state after each step.

Step	x	b	Action (inline or first OFA slot)	Snapshot (bucket b inline / OFA change)
1	23	2	inline	bucket 2: [23, □]
2	43	1	inline	bucket 1: [43, □]
3	13	6	inline	bucket 6: [13, □]
4	27	6	inline	bucket 6: [13, 27]
5	5	5	inline	bucket 5: [5, □]
6	25	4	inline	bucket 4: [25, □]
7	15	1	OFA at ofStart(15) = 1 (first free)	bucket 1 full; OFA[1] ← 15
8	35	0	inline	bucket 0: [35, □]

1. Diagram (5 pts). Use parameters $B = 7$, $c = 2$, $OF = 11$ and the insertion order

$$[\,14,\ 21,\ 28,\ 35,\ 7,\ 24,\ 31,\ 1,\ 8,\ 15\,].$$

Recall $h(x) = x \bmod 7$ and $\text{ofStart}(x) = h(x) \bmod OF$.

1. Build a *step table* with one row per insertion. For each key x, include: (i) $b = h(x)$; (ii) whether x is placed inline in bucket b (and which of the two slots), or in OFA; (iii) if in OFA, the *first* OFA index used (linear probing from $\text{ofStart}(x)$, wrapping modulo OF).
2. After the final insertion, draw the arrays:
   - All B buckets, each with two inline slots, labeled with bucket indices $0..B-1$.
   - The OFA array of length OF, labeled with indices $0..OF-1$.
3. Your table format should mirror the illustrative step table from the problem statement (columns for Step, Key, b, Action, and a brief snapshot comment).

---

2. C++ Program (9 pts). Fill out the C++ functions below. Behavior must match the rules above (OFA linear probing starts at $\text{ofStart}(x) = h(x) \bmod OF$ and probes $+1 \bmod OF$).

```
#define BB   7        /* number of buckets */
#define C    2        /* inline capacity per bucket */
#define OF   11       /* overflow area size */
```

```
#define EMPTY  -1
#define DEL    -2

/* Storage */
extern int Buck[BB][C];   /* -1 empty; each bucket has C slots */
extern int OFA[OF];       /* -1 empty, -2 tombstone */

/* Hash to bucket [0..BB-1] */
int bh(int x);            /* returns x % BB (non-negative) */

/* Overflow start index [0..OF-1] */
int ofStart(int x);       /* returns (bh(x)) % OF */

/* Initialize Buck and OFA (EMPTY) */
void hyInit();

/* Insert x: first try Buck[b]; if full, place into OFA by linear probing
   starting at ofStart(x); wrap modulo OF; return 1 if inserted, else 0 if OFA full */
int hyInsert(int x);

/* Search x: scan Buck[b]; if not found, linear probe OFA from ofStart(x)
   until EMPTY encountered (not counting DEL as a stop); return 1 if found else 0 */
int hySearch(int x);

/* Optional: Delete x: if x in Buck[b], clear that inline slot to EMPTY.
   Else, probe OFA and, if found, mark that slot DEL; return 1 if deleted else 0 */
int hyDelete(int x);

/* Optional: total number of OFA cells occupied (not EMPTY or DEL) */
int hyOFALength();

int main() {
    int keys[] = {23,43,13,27,5,25,15,35};
    int i, n = 8;
    hyInit();
    for (i = 0; i < n; i++) hyInsert(keys[i]);
    // print buckets and OFA; test a few searches and deletes
    return 0;
}
```

---

3. Complexity (3 pts).

1. Let p be the probability that a search lands in OFA (i.e., its home bucket is full at query time), and let L be the average *successful* probe length in OFA under the current load. Give the expected time of `hySearch` in terms of p and L.
2. Give the worst-case time of `hyInsert` in terms of B and OF.

4. Discussions (3 pts).

1. **Exact counts for this sequence.** For the sequence $[23, 43, 13, 27, 5, 25, 15, 35]$ with $B = 7$, $c = 2$, and $OF = 11$, how many keys end up in OFA? Give the unique integer.
2. **Invariant for correctness.** State a precise invariant that must hold between the bucket index $b = h(x)$, the OFA probe chain starting at ofStart(x), and the membership of key x after any sequence of inserts and deletes. Your invariant must be strong enough that, if it holds, `hySearch(x)` is guaranteed to find x if and only if x is present.

Problem 4: Perfect Hash for a Static Key Set **[20 points]**

You are given a static key set $S = \{10, 22, 37, 40\}$ and a table size $m = 7$. Consider the linear family:

$$h_{a,b}(x) \;=\; (a\,x + b) \bmod m \quad \text{with } 0 \le a, b < m.$$

Your task is to find parameters (a, b) such that $h_{a,b}$ is a **perfect hash** on S: all keys in S map to *distinct* buckets in $\{0, \ldots, m-1\}$. You will (i) implement a checker for injectivity on S, (ii) brute-force (a, b) to find at least one valid pair, and (iii) build/search in the resulting table.

How to reason about trials. We list a few trials to illustrate the process. A trial (a, b) computes the 4 bucket indices; if any duplicate appears, that trial *collides*.

Trial (a, b)	Buckets for $10, 22, 37, 40$ under $h_{a,b}$	Collision?	Keep?
(0,0)	(0,0,0,0)	yes	discard
(1,0)	(3,1,2,5)	no	candidate
(2,0)	(6,5,4,3)	no	candidate
(1,1)	(4,2,3,6)	no	candidate
$\cdots$ continue until you find *any* collision-free pair (a, b)			

Any candidate row (no duplicates) is acceptable for building the table.

---

1. Diagram (5 pts). Let $S' = \{3, 8, 12\}$ with $m = 5$. Produce a short trace table like the one above:

1. Try $(a, b) = (0, 0), (1, 0), (1, 1), (2, 0), (2, 1)$ *in that order*. For each trial, list all bucket indices for the keys of S' and mark whether a collision occurs.
2. Pick the first collision-free (a, b) from your table, then draw the final table of size $m = 5$ with each key placed at $h_{a,b}(x)$. Label the indices 0..4.

---

2. C++ Program (9 pts). Complete the following items. Your `isPerfect` must check injectivity *only over* the fixed S; `findPerfect` may brute-force (a, b) in any order.

```
#define PM 7
/* Fixed static set */
extern int SKeys[4];   /* = Example: {10,22,37,40} */

/* Return 1 if h_{a,b}(x) = (a*x + b) % PM maps SKeys injectively into [0..PM-1];
   otherwise return 0. */
int isPerfect(int a, int b);

/* Brute-force search over 0 <= a,b < PM. If a perfect pair exists,
   write it via *pa and *pb and return 1; else return 0. */
int findPerfect(int *pa, int *pb);

/* Build table T[PM] with -1 for empty; place each key x at T[h_{a,b}(x)]. */
void buildPerfect(int a, int b, int T[]);

/* Search key x in T using the same h_{a,b}; return index in [0..PM-1] or -1. */
int pSearch(int a, int b, int T[], int x);
```

```
/* ------------- The template for main() -------------

int SKeys[4] = {10,22,37,40};

int main() {
    int a, b, ok;
    int T[PM];
    ok = findPerfect(&a, &b);
    /* assert ok == 1 */
    buildPerfect(a, b, T);
    /* test: pSearch(a,b,T,22), pSearch(a,b,T,37), ... */
    return 0;
}
```

---

3. Complexity (3 pts).

1. What is the worst-case time to brute-force all (a, b) pairs (as a function of m)?
2. Once h is fixed, what is the time to search for a key x in the built table?

---

4. Discussions (3 pts).

1. **Counting valid parameters.** For $S = \{10, 22, 37, 40\}$ and $m = 7$, how many ordered pairs (a, b) with $0 \leq a, b < m$ produce a perfect hash on S? Give a single integer. (Your answer should be uniquely determined.)
2. **Characterizing injectivity.** Let m be prime, and suppose $a \not\equiv 0 \pmod{m}$. Prove or disprove the statement: for any distinct $x, y \in S$, the condition $h_{a,b}(x) \neq h_{a,b}(y)$ holds *for all* $b \in \{0, \ldots, m-1\}$. Your answer must be "True" or "False," with a 2–3 sentence justification that relies on modular arithmetic (no hand-waving).

Problem 5: Top-K Frequent Elements **[20 points]**

Given an integer array A (values in range [0..9999]) and an integer K, return the K elements with highest frequency. You will implement two components:

1. A frequency table using **open addressing (linear probing)** on fixed arrays.
2. A size-K **Min-Heap** storing pairs (freq, key) to track the current top K by smallest frequency at the root; when a new pair has strictly larger frequency than the root (or ties broken by a deterministic rule), it replaces the root.

Illustrative example . Let $A = [1, 1, 1, 2, 2, 3]$ and $K = 2$. We first build a frequency table, then feed each (freq, key) once into the Min-Heap with capacity K. The heap key is ordered by (freq, key) ascending; ties on frequency are broken by smaller key first. The table shows the heap after processing each distinct key.

Step	Pair (f, k)	Heap (as multiset of pairs)	Action	Root after action
Init	–	$\{\}$	–	–
1	$(3, 1)$	$\{(3, 1)\}$	push	$(3, 1)$
2	$(2, 2)$	$\{(2, 2), (3, 1)\}$	push	$(2, 2)$
3	$(1, 3)$	compare to root $(2, 2)$	no replace (since $1 \not> 2$)	$(2, 2)$
Final heap holds the top-K keys by highest frequency: keys $\{1, 2\}$.				

1. Diagram (5 pts). Use $A = [7, 7, 1, 1, 1, 4, 4, 4, 4, 3, 3, 3]$ and $K = 3$.

1. Show the final frequency counts for each distinct key (any order).
2. Provide a *step table* (same format as above) that shows the heap content immediately after processing each distinct key's pair (f, k).
3. Draw the Min-Heap *as an array and as a conceptual tree* after all pairs are processed.

---

2. C++ Program (9 pts). Complete the bodies of the following items.

```
/* ---- Hash table for frequency: linear probing ---- */
#define HM    20011   /* capacity */
#define HEMP  -1      /* empty marker */
#define HDEL  -2      /* tombstone marker */

extern int HKey[HM];  /* key */
extern int HVal[HM];  /* frequency */

/* Initialize the hash table arrays (HKey <- HEMP, HVal <- 0) */
void htabInit();

/* Increment frequency of x; insert x if absent; return index of x in table */
int htabInc(int x);

/* Find x; return index or -1 if not found */
int htabFind(int x);

/* ---- Min-Heap of (freq,key) with capacity K ---- */
#define TMAX 10005

extern int HKFreq[TMAX];
```

```
extern int HKKey[TMAX];
extern int hkSize;  /* current size */
extern int hkCap;   /* capacity (set by hkInit) */

/* Initialize heap capacity to K and reset size to 0 */
void hkInit(int K);

/* Push (f,k) if size < hkCap; otherwise if (f, k) should be kept while root
   is the smallest by (freq, key) ascending, replace root with (f,k) then heapify down.
   Otherwise, do nothing. */
void hkPushOrReplace(int f, int k);

/* Pop the min pair (write to *pf, *pk); return 0 if empty, else 1 */
int hkPop(int *pf, int *pk);

/* Heapify helpers */
void hkDown(int i);
void hkUp(int i);

/* ---- Driver ----
   Build the frequency table from arr[0..n-1], then feed exactly one (freq,key)
   pair for each distinct key into the size-K heap using hkPushOrReplace.
   Finally, pop up to K pairs into outKeys[] in descending frequency
   (break ties by ascending key). Return the number of keys written (<= K). */
int topKFrequent(int arr[], int n, int K, int outKeys[]);

/* ---- Expected program flow ----

int main() {
    int A[] = {1,1,1,2,2,3};
    int n = 6, K = 2, out[2];
    /* 1) htabInit(); scan A, calling htabInc(A[i]) for each i
       2) hkInit(K); iterate over table to find each occupied key k with freq f,
          then call hkPushOrReplace(f, k)
       3) Pop results into out[] in descending frequency order
    */
    /* print out[] */
    return 0;
}
```

---

3. Complexity (3 pts).

1. Under uniform hashing, give the expected time to compute frequencies for n items.
2. Let d be the number of distinct keys. After the table is built, each distinct pair (f, k) is fed exactly once to the size-K Min-Heap. Give the total time to maintain the heap in terms of d and K (big-O).

---

4. Discussions (3 pts).

1. **Determinism with ties.** Suppose the heap ordering is by (freq, key) ascending and the final output must list keys in *descending* frequency, breaking ties by *ascending* key. Is the final set and order of returned keys independent of the order in which distinct pairs are fed to the heap? Answer "Yes" or "No" and justify in 2–3 precise sentences.
2. **Exact operation counts.** Let d be the number of distinct keys. In the prescribed approach (compute all frequencies, then feed each (f, k) once), how many calls to hkPushOrReplace are made, and how many pops are used to produce the output array of size up to K? Give your answer as closed-form expressions in d and K.